

--

Practical Bug-Finding for Language Bindings across Package Managers

0 Meta (not the body)

sw: stale snapshot See:

the draft says (09-03)	now	re-derive with
12 agreements, 5 wired, under slugs such as <code>symbol_exported</code> and <code>soname_denotes_needed</code>	14 registered agreements: 11 with an evaluator, 9 landed . With 16 candidates, 30 claims sit on the model. Slugs renamed, e.g. <code>required_symbols_exported</code> , <code>soname_matches_requirement</code>	<code>canary checks --landing</code>
a five-pass pipeline	six passes over four IRs	<code>canary emit <p> --stage N</code>
10 projects and tiny1, 42 scenarios, 41 run	the record holds 10 projects, 28 worlds, 42 chains; z3 is muted and runs separately	<code>canary overview --json</code>
z3's forward cell: 791 symbols required, 705 provided	the notes disagree: 791/705 (08-19) and 776/705 with 85 missing (09-15). Run z3 and read its stub inspection before citing	<code>canary action z3</code> , then the record
web results page not built	the overview page is the results page	<code>make view</code>
the agreement catalogue doc (2382 lines)	retired; page §2 is the catalogue. Four agreement docs remain, due a cleanup	—

Status snapshot — 2026-09-03. Working-draft furniture; delete before submission. Baseline from the 2026-08-26 progress review; the checking row rewritten 09-03 against the agreement layer as it now stands. Percentages are judgement, evidence is not.

Track	State	Evidence
Framework (canary core)	~80%	M1 closed 2026-08-16. M2: steps 1–3 done, 7 of 10 open. Five-pass pipeline with <code>emit --stage N</code> dumps (08-24); own opam switch, platform carried not sniffed (08-26); four test suites green on Linux and macOS
Checking (the	~45%, moving	Now its own layer, <code>src/canary/agreement/</code> (10 modules, ~2.4k lines), replacing <code>surface/</code> — and it has production consumers (step builder,

Track	State	Evidence
agreement registry)		local runner, GH backend, project specs), closing the standing gap that it was a producer nothing read. Registry: 12 agreements — 5 wired, 1 blocked, 2 stubbed, 4 proposed , under named slugs (symbol_exported , soname_denotes_needed , ...) that replaced c1..c8 . Catalogue §1.7 is broader than the registry: 38 checks — 17 wired, 7 declared-but-unimplemented rows, 14 proposed — over three targets, each with a falsifier, a method, and a <i>source</i> . Doc is 2382 lines on a 12-section outline; §1, §2, §12 and §3.1 done , resuming at §3
Witness (tiny)	~85%, regressed	tiny1 22/22 pass, but detection coverage 12/24 — watchlist-blind on c5/c6/abi. tiny-full advertises six worlds and runs one ; its lib and binding axes sit in dead code. Open decision: restore or delete
Projects (empirical breadth)	~45%	10 registry projects + tiny1, 42 scenarios, 41 run. Against the declared 2×2 lower bound: 2 full, 1 collapse-only, 6 half, 1 neither
Findings (the product)	~50%	Real and reproducible: z3's forward cell (791 symbols required, 705 provided), ncurses libtinfo closure-shape segfault with identical symbol sets, zstd symbol-count-as-packager-policy, sundials 6→7 API break. Zero upstream PRs filed
Paper	~20% prose, spine settled	Old manuscript retired to draft_old.md (873 lines, largely roadmap bullets). This file is a bullet skeleton: claim agreed, three tiers agreed (enumerate / realize / attribute), materials triaged with a fate per file, related work surveyed. Prosing.
Delivery / ops	~25%	CI still runs the pre-A5 shape, one chain per project rather than the enumerated set; web results page not built; report milestone deferred

The through-line. The runner is the finished half and the checker is not: landing a project is cheap, but what a landing *checks* is still per-project tables, and no repair has yet been driven off a check. Growing the roster adds rows, not claims.

1. The problem

The claim. This is a bug-finding framework, not a verifier: it does not prove a deployment sound, it exhibits real failures in one. It works in three operations — **enumerate** the worlds a binding is actually deployed into, **realize** each one while keeping the record of how it was built, and **attribute** a violation back to the declaration or transition that produced it, with the agreement registry supplying the checks applied inside each world. The evidence is real defects, reported and fixed upstream.

Thesis: a language binding is assembled by many parties out of many tools, each of which checks its own step and takes the rest on trust. So the agreements between steps are never checked by anyone, and a defect made in one place is found in another, long afterwards, and charged to the wrong party.

- Multi-language bindings are ubiquitous and critical, and the chain that delivers them crosses tools and people no one specified together.

- Defects are therefore quirky, surface late, and get blamed wrong.
- **The chain spans actors:** the upstream library developer, the binding author (on-tree or off), maintainers in several package managers, an administrator, and an end user. Each sees one hop, and no one sees the chain.
- **Packaging is where those actors meet.** It is what delivers and distributes the software, so it is also where their cooperation has to hold. The end user starts from a *package* — never from the source the developer wrote, and rarely from the combination the binding author tested.
- **The chain rests on a mixture of agreements.** A few are specified; more are conventions; many are purely **behavioural** — true only because a compiler, a linker or a loader happens to act that way. None of them is owned by the party on the other side of the join.
- **The only oracle in use is a successful command**, and it is weak twice over. It is narrow: violations that some tool enforces get rejected, while everything merely *tolerated* passes green until usage finally touches it. And it is unfaithful: a linker may silently drop a version script, a build may not re-run, an install may skip a rule — all of it reported as success.
- **A defect is a property of the package *and* the use.** Different end uses touch different parts of a surface, so a deployment that is healthy under one user's usage is broken under another's. This is why checking has to reach the run and cannot stop at the build.
- **Management latency puts distance between cause and symptom.** The packaged binding lags the library it wraps, so a defect enters upstream and surfaces downstream — long after shipping, and far from the party who could fix it.
- Together these make binding packages **error-prone to produce, hard to test, and hard to attribute** once something breaks.

What already exists, component by component

- **The native library.** Consumers rest on its exported symbols, its soname, its version nodes. This is the best-served component: **abidiff**, **abicompat** and the newer **abichk** compare a library against a consumer, and Debian's symbols machinery and Fedora/openSUSE ABI QA run such checks routinely. They compare the two binaries they are handed, and never ask which two a user will actually have.
- **The foreign interface** — the binding's stubs and declarations. Per-language analyses exist (Jinn, TurboJet, FFIChecker, Python/C checkers), as do the generators (SWIG, **bindgen**, cffi). Each covers one language's protocol, and a generator validates against the header it was given rather than the library that will be loaded.
- **The binding's user-facing API** — what the repacking presents upward. Essentially only the binding's own test suite covers it, and a test suite exercises what its author thought to exercise, in the one configuration its author had.
- **Versions and dependency constraints.** Every ecosystem ships a resolver — opam, pip, conda, dose3 — and cross-ecosystem resolution now has research systems too: HyperRes, and the Package Calculus. The deployed resolvers decide what *may* be installed together from **declared metadata**, never whether the installed artifacts fit; the research systems model resolution and stop, deliberately, before anything is realized.
- **Where each artifact came from.** SBOM formats, SLSA and reproducible builds record provenance faithfully. Provenance answers *where it came from*, which is a different question from *does it fit*.
- **The realized deployment, and the run.** Ecosystems run substantial rebuild-and-test infrastructure — autopkgtest, conda-forge migrators, opam bulk builds, **cibuildwheel**, **auditwheel**, **rpminspect**.

Each explores within one ecosystem, at the versions that ecosystem chose, normalising foreign dependencies into its own package model.

- **The two package managers, by side:** apt and brew on the system side, opam and pip on the language side (page §3.1).
- **Where they meet:** a bridge, or nothing (page §3.3). opam's conf-* packages and depexts are bridges; pip bundles the library instead. Cargo's *-sys crates meet through a capability file and no bridge. That contrast left the page and belongs here or in §8.
- **Every partial answer stops at the same boundary: it takes the other side as given.** The library checker is handed its consumer; the FFI checker is handed its library; the resolver hands over metadata and stops; the rebuild farm fixes the ecosystem and varies inside it. The defect lives in the join, and the join is the one thing no party owns.
- **The asymmetry is one of adoption, not only of ideas.** Everything that became production infrastructure is *local* — auditwheel and manylinux, cibuildwheel, opam depexts, conda-forge migrators, Debian archive QA, rpminspect, libabigail. Everything compositional is a prototype or an emerging standard. A correct compositional check therefore helps nobody today, because nothing runs it in the world the user gets.
- **So a binding can fail while every local subsystem behaved correctly by its own contract.** The questions nobody asks are the compositional ones: which header did the binding actually observe, which implementation did its package manager intend, which provider did the build select, which one did the runtime finally load, and does the declaration the binding carries match that provider?

Material: draft_old.md §Motivation; the courtesy paragraph; related/canary-practical-cross-language-bindings-report.md §§2–5, 9–10; related/adoption-in-practice.md (the adoption asymmetry).

2. Our solution, and why it wins

2.1 Rationale: perspectives and solutions

The bug is tricky. The trickiness come from two parts, one is the essential complexity in the problem setting, which is a combination of two languages, which are across the binding mechanism, and managed by two package managers; also, each concrete case also depends on the specific package manifest and the running environments.

Our target is to study the workflow a.k.a real-world actions around the language bindings. It doesn't only cover immediately on how an binding is created and used, but also cover all the lifecycle for the dependent artifacts, tools including package managers, and systems where they are working on.

Given many involved artifacts and tools are real-world ad-hoc usage and solutions, we are lean to the practical analysis, which starts from the *actions* in real-world, and identify the *agreements* people wish to obey, and confirm or blame as experienced programmers. We don't start from the specification of tools or artifacts, and we also don't target to provide any formal definitions or semantics.

We choose the practical approach, that experienced programmers may use to trigger real-world bugs, that is to construct sanity check programming and run tools or commands. When treating it as a test generation issue, two questions are how to generate tests, and how to expect the results.

The motivation to enumerate actions, as the analogy of test programs is well-explained in before, because no whole knowledge of package around bindings are exploration by any user groups.

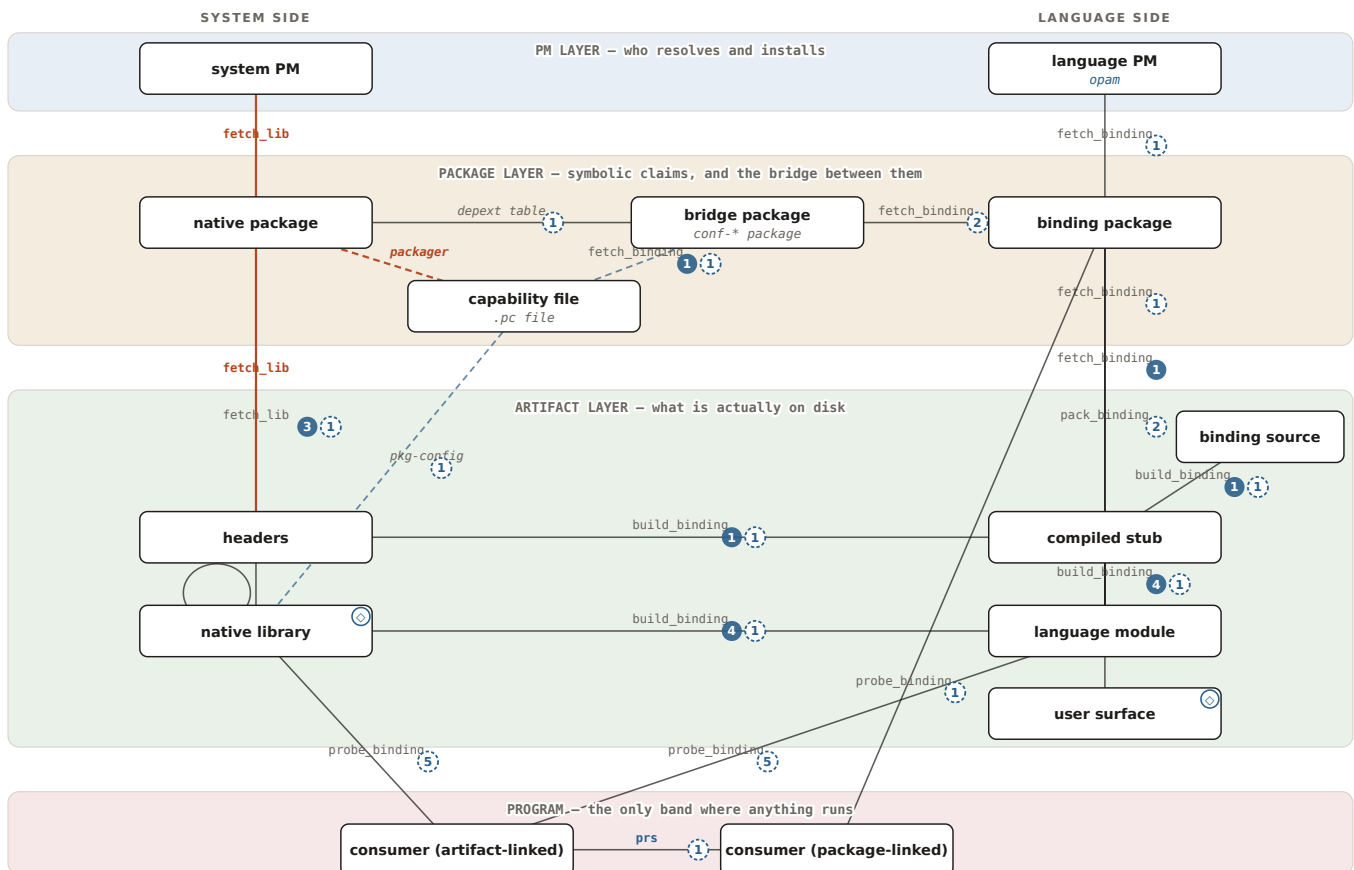
$$PM_{sys} \rightarrow Package_{sys} \leftrightarrow Artifact_{sys} \leftrightarrow Binding \leftrightarrow Artifact_{lang} \leftrightarrow Package_{lang} \leftarrow PM_{lang}$$

Seek for expectation is challenging for this herotegenous configurations. For example, if a user encounters an misaligned function, who shall it blame, himself, the binding package, the system package, etc? We establish the perspective, which is also a theoretical model to understand the combination for those components. Aside, this perspective also carry a diagram template to show both the generic workflow and any concrete running logs.

sw: missing package special irregularity

At the artifact-layer, we treat binding mechanism as first-class variant to study, while at the package-layer, we also treat package managers' cooperations explicitly. This split also helps to split normal managements on naming, versioning, resolution, dependencies, and also give the cooperations a dedicated study, that interaction based on package content, and via special constructs e.g. virtual packages. We can see here naturally residents a M*N problem essentially, and it cannot always be solved by the courtesy from some good packages.

2.2 Layered View for Actions



We are seeking a principled framework that helps to understand, reason, and explain the issues around packages for language bindings. We propose to split the workflow into package managers without package content, cooperations of package managers, certain package customization, creation of package content, binding mechanism across two languages or file formats. *sw: rewrite the part* Roughly, we ascribe some to the resolution of resource to package managers, and some to the producing and consumptions of artifacts.

The perspective helps to reason the ubiquitous irregularities of package usage in a uniformed generic framework.

We propose a layered model, shown in the diagram. The diagram depicts the package managers, packages, and artifacts from system and from language in each side. The diagram also shows the layers for package managers, packages and artifacts, and programs. The model as well as the diagram has benefit to show irregularity packages. To run a complete binding, there must be artifact in the system side. If that artifact is not managed by system package manager directly, we can still see the artifact as its designed place. It can cover cases e.g. bundled in same language side package, provided by another language side package, or provided by a virtual package, etc. The model also places the aligned parts in each side horizontally, thus we can see the matching packages, source code, interface files, and compiled artifact if having.

sw: some words for an example from the diagram here

artifacts have its own rules. No matter for system package managers who maintains native libraries, or language package managers who maintains language packages, we can find any artifact understood managed, including ELF-format object, or OCaml or Python modules, are agnostic by package managers. Creation, usage, and inspection of these artifacts are provided by binary or language tools, rather than any package managers.

package managers are not coupled. Different linux distro can have separate package managers, for example, we can have Debian's apt and Fedora's rpm. We see new package managers for Python or JavaScript invented from time to time. Even for OCaml where opam is the official standard package managers, we can have *dune package management*. Regardless of whether being official or de-facto software, from the design space, a package manager is external to the artifact being packed and managed. *Familiarity is not true knowledge.*

package can almost customize anything. Some artifacts in a package may need a complex and subtle dependency and environments. The package manager may not have features, or the package registry is not satisfiable, or they are not working well at that moment when that package is published or maintained. The good thing is package managers often let a package rescue itself with shell script, patches, external resource fetch, however, it may also bring complexity at the same time. *Extraordinary dependencies require extraordinary customization.*

Aside: composition of multiple rule sets with mixture stages Common PL tools are concerning one set of rules, and the combination of applying them on joinable components, e.g. interpretation, typecheck, static analysis, etc. Package managers under our discussion, is the real-world application, to combine a group of different set of rules, that covers multiple package managers, multiple package content, and the cooperations between them. Some sets of rules can be typechecker, e.g. the provider or consumer of the bindings can be in a typed language, but it's just some cases. They also don't always start with the original input, say even with the full language tools, we cannot re-compile and re-build everything from scratch. The problem extended to ensure the arbitrary artifacts

Aside: variants of (lambda) calculus. In PL, there are variants of languages, e.g. pure lambda calculus, and lambda calculus with support or dissupport of natural numbers, integer, booleans, and any operations in and between them. Some entities and their operations are algebraic and exist ahead of the computational model. We can also have other computational devices for example, first-class language. This opens a perspective to see a concrete language is consisting of the computational side but not doing algebraic

operations, and the algebraic (logic) side. The former provides the variables, substitution, resolution, closures, which all assist in computational, and can be composed to others. This is an analogy of the package management side, and the artifact inside of a package.

2.3 Canary Overall

Thesis: a project declares only what it is; the framework derives the worlds, realizes them, and keeps the record of how each was realized. The deployed set is a product, and a product is derivable — which is what makes provenance an experimental variable rather than a fixed assumption.

This work presents the Canary framework. Canary enumerates practical operations, from a project manifest containing the necessary project-specific commands for building, installing and delivering the project, together with declaration of artifacts such as source code, packages and releases for upstream libraries or binding. Each enumerated scenario contains a sequence of possible actions that may span several stages. A scenario can run locally through an OCaml driver program, or as a continuous integration workflow. Scenario enumeration provides coverage for situations in which artifacts of interest may be used, in situations that library developers or package maintainers may not anticipate and that library users may not encounter exhaustively.

Besides enumerating scenario combinations, we also enhance each scenario's action chain with checks dispatched from the agreement registry for the involved artifacts. For example, consider a project `tiny`, in which an OCaml module uses a binding to a C library. Its source files include the header file `tiny.h`, C source `tiny.c`, OCaml C stub `tiny_stub.c`, OCaml binding `tiny.ml` and an OCaml signature `tiny.mli`. Building OCaml binding `dlltiny.so` and others needs C header and the compiled library `libtiny.so` (assume ELF format in Linux). Compiling and running the OCaml binding locally needs linker and loader to treat `dlltiny.so`. With the binding, a user can use it directly, but a library developer may publish `tiny_help` library which declares this dependency, and let the user fetch that file. Both system and language-specific package managers help deliver and resolve artifacts. However, they also complicate the situations on how artifacts are declared, which artifact to use, whether they fit together, and more broadly, how the package managers cooperate.

3. Model of Package Managers for Language Bindings

In modeling the workflow into several components, we list and discuss each component in following subsections. *sw: the material should have been sync-ed with code, but not audited yet.*

3.1 Package Managers for its Management part

@import "exhibits/tab-pm.html"

- The part is on primitives for package managers including stores, (syntactical) versioning, resolution.

The package side can resolve to a required package and to fetch the package material. The most safety we expects are resolvability. It includes the naive package resolution, payload related resolution e.g. path in ELF, which needs discussion in 3.2, and cross package managers resolution, in 3.3

3.2 Package Payload Integrity

This subsection concerns the properties of the content that is under managed. They include the properties that the payload artifact shall obey, and also serves a part what an agreement can only be checked jointly.

3.3 Package Manager Cooperations

@import "exhibits/tab-cooperation.html"

3.4 Binding Mechanisms

@import "exhibits/tab-mechanisms.html"

- The part lists both sides of a binding mechanism. Both sides keep its own integrity
- The part is package-free

Initially introduce *admissible substitution*: practically, one cannot rebuild everything from the source, and the outer resolution side can provide an arbitrary artifact.

4. Practical enumeration

4.1 RATIONALE — why enumerate at all

The motivation for enumeration is straightforward: **to dry-run possible scenarion and encounter the error before users do**. Some errors or bugs only occur in particular scenarios. For example, a build script may set a incorrect linker flag in creating the release version of a library, and the bug may survive the subsequent binding building. Detecting such bugs before users report them can reduce their impact on users and the maintenance burden. The problem comes down to **what** to enumerate and **how** to enumerate it.

Canary's enumeration is a **practice choice**. It is neither intended to explore every possibility exhaustively nor limited to combinatorial testing over fixed constants. Canary intends to permute the important artifacts, including official source code, building artifacts and packages in official registries. As a heuristic, we try at least two versions of each artifacts: the latest stable version and a development version at the time of checking.

One way that canary handles real-world version irregularity is by **bypassing existing restriction and anticipating combinations**. Real-world projects often have uneven release schedules. Development may be active while releases in package registries lag behind. Tensions between dependencies are intrinsic, regardless of the software or versioning scheme used. If the dependency remain fixed to old components, old bugs can never be repaired; if the dependency is not kept, compatibility can never be guaranteed.

To realize these version combinations, Canary provides customized package manifest file for each package manager, and makes a local package storage. This allows us to provide packages for development versions even before an upstream package is released. We also modify the existing manifest files of packages to allow dependencies on development versions. It's a radical modification but often works, because many packages for bindings usually just boost their dependency then encounter the same situation needing fix as we can encounter with this modification.

Real-world packages often bundle its dependency, especially for artifact not managed by the same package manager, e.g. pip's `z3-solver` bundles a `libz3` in the Python package. `opam` also have packages named `lib<pkg>`. It's a fragiled but common practice. Canary handle real-world provisioning irregularities by supporting ad-hoc package *co-providers* to specify them in our project specification.

4.2 Action, the basic operation and enumeration unit

Canary uses actions to describe project operations, including *build-library*, *build-binding*, *install-library*, *publish-package*, etc. An action's input and output components are statically known. Some action outputs become the input to other actions, e.g. both *build-library* from source and *fetch-library* from a package manager can provide a library consumed by *build-binding*.

The actions form a directed acyclic graph (DAG), with nodes representing actions, and edges representing there are artifacts passed between them. Duplicate actions can be represented by the same node. Dependency resolution for action enumeration is simpler than resolution in build systems or package managers, because the graph can be constructed statically. It does not require dynamic decisions about which binding or mechanism to use.

Action enumeration has two parts: the action shape, in which the provenance, version, platform and other choices are not yet specified, and the concrete choices for those parameters. It's quite like enumerating arithmetic expressions of a given length: expression shapes (choices of operators) and numbers (choices of operands) belong to different categories.

Action-chain shapes can be enumerated independently of the concrete projects being tested.. It also reflects the common structure of project workflows. The project-independent enumeration can be shared, while only project-specific information needs to be declared in a dedicated data structure.

4.3 Project Manifest

(SW: *spec* may not be a good name. The intention looks like a project *manifest*)

In the Canary framework, a project such as **z3**, **llvm** or **torch** is described by a project specification. The specification declares the project-specific commands that actions may need, such as commands for building, installing and testing. Most of this information can be collected from the project's official documentation. It's one of the few places of Canary that requires human effort to audit.

A project specification also declares an arbitrary number of resources as artifact providers. Canary provides utilities for fetching from common package managers, remote resources and local vendored sources. The specification is a static declaration, and a following collection pass gathers providers for each artifact. For example, the providers of native library **z3** include official project source, which needs to build the latest stable packages in the platform of interest, and the python package **z3-solver**.

4.4 Running Actions

An action is instantiated using information from a project manifest. For instance, the *build-project* action for project **z3** gets the building command and the path to the source code artifact. Canary also prepares the output path required by the build command.

Canary supports running the actions on a local machine for quick execution, and on CI backends, such as GitHub CI, for a persistent and reliable records. Supporting both local machine running Ubuntu or MacOS and backend which primarily takes a YAML file involves subtle design considerations.

The local execution target is an OCaml program that invokes external tools and shell scripts. Actions and artifacts have unique identifiers, helping minimize unnecessary repetition. Actions whose inputs and outputs consist only of files can be cached naturally. For package managers that do not allow multiple versions of the same package name to coexist, action requiring different versions need fully *uninstall* one version then *install* another, or use separate package environments, which are currently often expensive for OCaml.

Targeting GitHub CI YAML files involves additional design considerations. A YAML can contain multiple jobs, and each running on an isolated machine. Jobs naturally share nothing. We assign each job to hold a list of action (a path in the action diagram), so later actions can reuse the earlier result. Currently, some actions are duplicated across jobs. The advantage is that, as in common usage of GitHub CI usage, a job's status reflects the status of a complete scenario, e.g. *fetch-project-v1*, *build-project-v1*, *building-a-binding-vb2-with-v1*, *pack-vb2-as-a-package*, *fetch-package*, *run-app-using-package*.

4.5 Compiling to Actions

Canary implementation approaches is like a compiler which takes static surface information as the initial IR, to runnable steps on concrete backends end terminal IR. This design helps to clearly inspect information in each stage, reduce duplicated work on configurations, and only fulfill complete commands until the platform-dependent instructions are necessary.

sw: t-b-c The necessity and benefit is mainly for engineering, a.k.a practical usage.

5. Principled checking

5.1 Rationale

PL researches usually concern one set of rules or semantics, and it applies on multiple components compositionally. Real-world package managers across language bindings contain naturally heretogenous parts, while some steps can be typechecked if one side of binding uses a language supporting it. Thus, it may combine multiple systems on integrating good virtue of each involved components that people wish to have.

The complete pipeline for a consumer program of using a binding can be split into three parts, resolving the binding side module, resolving the native side module, and invoking the binding mechanism. The real order of these parts can vary and differ, but we can see the different layers for the artifacts that around the bindings, and the peripheral structures and steps that helps to find those artifacts.

The **artifact layer** is dictated by the language and binary tools. The agreement between the inputs and the outputs is introduced by the tools, and indirectly by any specification if that tools obey. For example, a compiler compiles a source code `src` to an output `obj`. We can inspect the `obj` to find the evident that it should be aligned with the `src`: if it's the case that C code compiles to an ELF object, we can inspect the symbols, functions, etc; if it's the OCaml compiles to code to a bytecode, we can using ocaml's `objdump` to inspect. The observation here is the artifact creation carries some agreements from the tools and the creation is unavoidably to lose information. These agreements is a *search of the lost agreement* for arbitrary given artifacts, that the package managers might provide. With this perspective, for any existing actions in the artifact layer, we can seek some direct agreements.

Considering the binding mechanism, we can also find agreements that spans several actions. For example, the binding usually provides similar interfaces in different sides. For example, the native library can provide math operations in its syntax, while the binding library has corresponding operations. These agreements depends on which binding mechanism you are using. for example, whether requiring C stubs, whether involving separate compiled steps, or whether pure dynamically. Beyond the binding mechanism, both the

presense and invariants on one side shall be kept conventionally on the other side. The agreement can be detected and checked via either the static inspection, or dynamic runtime tests

The remaining pipeline are in **package layer**, which concerns how to declare and locate the artifacts as package payloads. Our would like to view one package manager from the package payload side and the management side, because they don't have to be coupled. For example, we usually describe Debian apt as a system package manager, and it uses to manage system packages, some of which are native binary libraries. We can see ELF has its own specification, versioning, conversions, which are not determined by the package managers. The same is also true for Mach-O and homebrew. However, the package manager side can enforces some conventions for the managed material, e.g. how to set its rpath. We treat the package management side and payload side separately, and it helps us to understand and propose agreements better.

The artifacts for a binding can be provided by separate package managers. Multiple package managers have different cooperation patterns. A package manager can be aware and even specify another package manager. For example, opam once uses **depext** to specify the external dependencies and now prefer to use virtual conf-package which directs depending on system packages managers. If a package manager itself is agnostic about other package managers, the task is either left to packages or to users, so that the solution (or the fact) are various. For example, opam's **z3** package currently has to build a **libz3** within the opam package. For pip's **z3-solver**, it ships a prebuilt **libz3** in the package.

The above disussion forms perspectives to treat the actions as the join from two package managers including their management parts and payload parts, and the binding mechanism parts. The four components at three layers helps to category the agreements.

5.1 Rationale (Old, Shall be absorbed)

There are several motivations for principled checking. (1) we want to make explicit which checks are performed and what they mean. For individual artifacts, such as a native binary, may contain dangling symbols, hardcoded path, mismatched functions that do not match a binding's expectations. Whether these properties are tolerable, resolved later, blame itself or the binding side, may depend variously on ABI specification, other tools, binding mechanism and conventions practice used by package managers. Enforcing every agreement is not practical task for a checking framework, but making the agreements explicit is feasible. (2) registering checks as pre-action or post-action hooks, or as special actions, allows them to be invoked uniformly to be invoked uniformly without relying on developers to remember to run them.

Information loss is intrinsic. For example, C the source code, including header file, is compiled to a native library. Later actions may use different versions of the headers and the native library, whose compatibility must then be checked externally. In ELF, linking records the linked library's name in the binary's **NEEDED** entries. the loader later searches for the library using that name to find an alternative, but whether the work as previous is unclear.

Any involved artifact can be published through a package manager, and a later action may use a different artifact fetched somewhere else. Whether a different artifact, such as a header file or library, is compatible with the previous one is a undecidable quesiton. No versioning scheme can encodes all the invariants.

Given that most of the checks are not strict requirements, we use *agreement* to describe the expectations that an artifact or an action should satisfy but can fail. We avoid stronger terms such as *invariant* because software can often still work even when an agreement is violated.

Canary centralizes the agreements in a module named agreement registry. When an action touches a particular artifact, or involves a binding mechanism or tool, it will look up the agreement it should check. Each agreement provides a command template, so that the corresponding can perform alongside with the action.

5.2 Agreement at a Glance

Agreements can be on one or several artifact. Artifacts can be checked by inspecting artifacts with tools or running tests. We use *bind mechanism* to denote the method by which a binding becomes available. Considering the common situation in which an upstream project provides C code that can be compiled into a native lib, and OCaml or Python code needs to use it. Both languages offer several mechanisms for using native libraries, including compiled C stubs and dynamic loading. The artifacts involved can be fetched from package managers or online resource, and users often rely on constrains like versions to select the appropriate artifacts. We would like to go a step beyond these common scenarios, which Canary can already enumerate, and ask which agreements we can expect and check.

what do we check against. t.b.c

how do we check. The checking follows our practical rationale, as experienced developers, we only use existing tools to inspect artifacts including system binary utilities and language own toolset.

what do we check.

5.3 Native Binary Artifact

Many upstream project provides C libraries, and other languages can access through the system ABI. Compiling a C library requires header files that may also be used later in building (but not using) language bindings.

Agreement between a header file and native binary is established during compilation. Given an arbitrary header and binary, however, that agreement must be recovered and established again. Violating it does not always cause a runtime error. A missing declaration may be tolerated if it is never needed.

Native binaries acquire information as they are compiled and linked. When native library is built for delivery using the *install* build profile, the agreement is that the binaries should not contain local path information. The violation may be tolerated if other conventional search paths compensate for it.

We use compiler toolset and platform binary utils to inspect the artifacts.

5. Attribution and Blame

Thesis: a red cell says something broke; a finding says who is answerable. The step between them is not a heuristic — the blamed party is read off the source of the claim, and the derivation kept during realization is what makes that reading possible.

- **The blamed party is read off the source of the claim**, not off the artifact that happened to fail. Each of §4's seven sources carries one: a self/format failure indicts the artifact; a declaration failure indicts *either* the artifact or a stale declaration, and the row must say which side it treats as authoritative; a peer failure indicts a side; a sibling-world failure indicts neither artifact but the **provisioning**.

- **A failing check means the artifact or the claim is wrong.** A row that does not name which it trusts produces an unattributable finding — which is a bug report nobody can act on.
- **Direction is a tiebreaker, not the rule.** It resolves a peer failure only when the two sides differ by *version*: forward, the consumer asked for too much; backward, the provider dropped something. When they differ by *packaging* — both artifacts correct, the versions drop-in compatible, the conventions disagreeing about how one implementation is named and divided — no direction exists, and the blamed party is the **cooperation**. ncurses is the specimen that forced this amendment.
- **Blame is an output, not a narrative:** the check names the surface, the surface names the actor, and the world's derivation names which decision put that artifact there.
- **What exists today, stated plainly.** Which *agreement* confirmed a failure is recorded per step and survives into the run record; direction is computed per scenario and displayed. Party-level blame is **not implemented** — the detection pass still reports only whether a step errored and whether its output appeared. The claim is therefore a *model with two working instances*, never a localizer.
- **The merit claimed is integration, not per-artifact strictness.** The specialist checkers are more rigorous about a single artifact than we intend to be. What none of them does is carry a world's derivation far enough to name a responsible party.

6. Implementation

7. Evaluation and Result

*Thesis (plan):

- We applies the canary on dozens of real-world projects and detect those bugs. Some of admitted and solved as PRs.
- We proposed several new bindings that are more confirmative than their counterparts.

8. Related work

Thesis: §1 already met this work component by component; here it is grouped by what each line contributes, and by whether it is a neighbour, a backend, or a rival.

- **Artifact-level checking** — libabigail (**abidiff**, **abicompat**), **abichk**, **abi3audit**, **auditwheel**, distribution ABI QA. The closest work, and the natural **backends**: they answer a pairing we hand them.
- **Cross-language FFI analysis** — Jinn, TurboJet, FFIChecker, Python/C static analyses. Per-language protocols, deeper than anything here on their own ground; we claim no new theory of FFI faults and would run them as backends too.
- **Dependency resolution** — the ecosystem resolvers, dose3, and the cross-ecosystem work: HyperRes, Package Managers à la Carte / the Package Calculus. They decide what *may* be combined, and stop before realization, deliberately.
- **Provenance and supply chain** — SBOM formats, SLSA, reproducible builds. Complementary: they record how an artifact arose, a record this work also needs and puts to a different use.
- **Ecosystem rebuild and QA infrastructure** — autopkgtest, conda-forge migrators, opam bulk builds, **cibuildwheel**, **rpminspect**. The nearest in *shape*; the difference is the unit of exploration, not oracle strength.
- **The theory side** — verified and type-preserving compilation, linking calculi, ELF and FFI semantics. Inherited: a behaviour check as refinement of an observable trace. Departed: those systems own every

pass, and we own none of them.

- **Adoption is itself a finding.** These lines divide sharply — package, build, binary-policy and ABI tooling reached production infrastructure, while cross-language and cross-ecosystem analyses remain prototypes or emerging standards. The composition gap is not only conceptual but deployed.

Material: related/literature.md (theory, 12 sections); related/canary-practical-cross-language-bindings-report.md (practice); related/adoption-in-practice.md (adoption). §2 carries the four objections; this section carries the map.